GitRekt — Demand and Production Forecasting Models

SmartHack 2026

Abstract

The battery agent decides every 15 minutes whether to charge, discharge or hold, so it needs to know how much energy will be consumed and produced. Two machine-learning models supply this: **ML 1** forecasts Romania's national electricity demand, and **ML 2** forecasts national solar, wind and total production. Each model gives a forecast for any 15-minute interval up to 7 days ahead, together with a 10–90 % range. This document describes the data, how it is collected and processed, how the models work, and how accurate they are on 2026, a year they never saw in training.

1 Data

1.1 Sources and collection

Energy (ENTSO-E). The ENTSO-E Transparency Platform publishes grid data for every European bidding zone. Two datasets were downloaded for the Romanian zone (`10YRO-TEL---P`) at 15-minute resolution, from 2022-11-07 to 2026-11-07:

- *Actual Total Load*: national demand in MW;
- *Actual Generation per Production Type*: nine sources (biomass, storage, lignite, gas, run-of-river hydro, reservoir hydro, nuclear, solar, onshore wind).

The platform exports at most 7 days per request, so `scripts/entsoe_export.py` and `scripts/total_load_day_` split the range into 7-day windows. Each window is registered as a download request and polled until the CSV is ready. Finished windows are cached on disk, so an interrupted run resumes where it stopped.

Weather (Open-Meteo). `predictions/build_dataset.py` downloads weather from Open-Meteo's *Historical Forecast API*. This API stores past *forecasts*, not measurements, so the training inputs look like what the live system receives. Weather is only collected for series that depend on it. Each variable is a weighted average over sites chosen for what it explains:

Variables	Explains	Sites (weight)
temperature, precipitation	demand, hydro	7 largest cities, weighted by population
shortwave radiation, cloud cover	solar	Olt, Giurgiu (1.0), Banat (0.5)
wind speed at 80 m	wind	Tulcea, Cogealac (1.0), Galati (0.5)

Nuclear, gas, lignite and biomass are controlled by plant operators, not by the weather, so no weather is collected for them.

1.2 Processing

1. **Time alignment.** The load file uses local time (CET/CEST, with special rows on the days the clocks change), and the generation file uses UTC. `scripts/build_dataset.py` converts both to the UTC start of each interval, stored as a Unix timestamp. It keeps only intervals present in both files, and turns placeholders (`-`, `n/e`) into empty cells. The generation file has one row per source, which is reshaped into one column per source.

2. **Derived targets.** `total_production` is the sum of the nine sources. `other_production` is total minus solar minus wind.

3. **Invalid rows.** 48 records have zero load, or zero output from every source that always runs. They are marked `valid = 0` and left out of training.

4. **Calendar.** For each interval, in Romanian local time: quarter-hour of the day, weekday, month, a sine/cosine encoding of the season, weekend flag and Romanian public holidays.

The result, `predictions/dataset.csv`, has **140,302 rows**, one per 15 minutes, with no gaps.

2 The models

2.1 What is predicted

Series	Model	Main drivers	Seasonal reference
`actual_total_load`	ML 1 (demand)	time, weekday, holidays, temperature	same time last week
`solar`	ML 2	sun elevation, irradiance, cloud	same time yesterday
`wind_onshore`	ML 2	wind speed	mean of last 24 h
`other_production`	ML 2	follows demand and the market	same time last week

Total production is solar + wind + other. The models output values in MW, for all of Romania. The simulation turns these national values into one site's values with a fixed factor per series (`SiteScale` in `loop/real_day.py`). For example, the 99th-percentile national demand of 8,429 MW becomes the site's peak of 4.5 MW.

2.2 One model for every lead time

Each forecast is made for a *target time* T from an *issue time* t_0, the "now" of the forecast. The *lead time* is $h = T - t_0 + 15$ min. Every feature uses only data from before t_0. This means the same code is used for training, testing and the live app, and none of them can see future data. Because h is itself a feature, one model answers both "15 minutes ahead" and "6 days ahead". The features are:

- **recent history:** the latest known value, its change over the last 15 min and the last 1 h, and the mean of the last 24 h;
- **lags:** the value at T minus whole days or whole weeks, using the most recent one already known at t_0;
- **calendar and sun:** the calendar of T and the sun's elevation;
- **weather:** the five weather variables at T;
- **lead time and base:** h itself, and the base forecast defined below.

2.3 Predicting a correction to a simple forecast

A tree model cannot copy one of its inputs exactly, yet 15 minutes ahead the last known value is very hard to beat. So the model does not predict the value y directly. It predicts a correction to a simple *base* forecast:

$$b = w\, y_{\text{last}} + (1 - w)\, y_{\text{seasonal}}, \qquad w = e^{-(h-0.25)/3}, \tag{1}$$

$$z = \frac{y/s - b}{\sigma(h)}, \qquad \sigma(h) = \sqrt{\min(h, 24)/24}. \tag{2}$$

All values here are divided by the scale s before use.

- The base b is exactly the last known value 15 minutes ahead. Further ahead it moves towards the seasonal reference in the table above.
- The **scale** s is a recent level of the series: the 7-day mean for demand and other production, the 30-day maximum for solar, and the 365-day maximum for wind. Trees cannot predict values larger than any seen in training, and Romania's solar peak in 2026 (3,395 MW) is almost twice the 2025 peak (1,845 MW). Predicting ratios lets the forecast grow with installed capacity.
- Dividing by $\sigma(h)$ puts every lead time on the same footing. Without it, long-lead errors, which are about ten times larger, would dominate training.

For each series, three **LightGBM** gradient-boosted tree models are trained with quantile loss, for the 10 %, 50 % and 90 % quantiles. The 50 % model gives the forecast; the other two give the range.

2.4 Training

1. **Sampling.** Each interval is paired with 4 random issue times between 15 min and 7 days earlier: half drawn log-uniformly (many short leads) and half uniformly. This gives about 440k samples per series.
2. **Early stopping.** The models are trained on everything up to 3 months before the cut-off date, and trees stop being added once the error on those last 3 months stops improving. Settings: 31 leaves, at least 1,000 samples per leaf, learning rate 0.05, row and feature subsampling.
3. **Refit.** The models are retrained on all data before the cut-off, using the number of trees found in step 2.
4. **Range calibration.** Quantile models are usually too confident on new data. The models from step 2 never saw the held-out 3 months, so their errors there are honest. For each series and lead bucket (0–1 h, 1–6 h, 6–24 h, 1–3 d, 3–7 d), a factor k is found so that forecast $\pm\, k \cdot$ half-width covers 80 % of the actual values.
5. **Totals.** The range of total production combines the ranges of its parts, assuming their errors are independent: $\text{half-width} = \sqrt{\sum_i \text{half-width}_i^2}$. It is then calibrated in the same way.

For evaluation the cut-off is 2026-01-01, so no 2026 data is used in training. The models the app uses are trained on all the data.

3 Results

The test set is **every 15-minute interval of 2026** (1 January to 7 November, 29,806 intervals). At each lead time, every interval is forecast as if it had been issued that far in advance. The models are compared with two simple rules: *persistence* (the last known value) and *weekly naive* (the same quarter-hour a whole number of weeks earlier). *Skill* is how much lower the model's error is than the error of the better of the two rules.

Lead time	15 min	1 h	6 h	1 day	3 days	7 days
Mean absolute error, MW (in brackets: as % of the mean value)						
Demand	49 (0.8)	109 (1.9)	218 (3.8)	237 (4.1)	263 (4.5)	291 (5.0)
Solar	18	44	89	87	90	92
Wind	22	75	202	200	198	201
Total production	74 (1.4)	189 (3.6)	358 (6.9)	354 (6.8)	380 (7.3)	394 (7.6)
Skill vs the better simple rule, %						
Demand	36	58	48	44	38	31
Solar	57	71	50	41	49	48
Wind	19	17	40	65	69	71
Total production	6	12	49	46	46	44

Table 1: 2026 backtest (full table: `predictions/reports/eval_2026.csv`).

Range coverage (target 80 %): demand 72–84 %, solar 81–86 %, wind 74–82 %, total production 69–76 %.

The models beat both simple rules at every lead time, for every series. A day ahead, which is the horizon used to plan against the day-ahead prices, demand is within about 4 %. Grid operators typically reach 2–3 %. Solar (about 3 % of installed capacity) and wind (about 7 %) are within the usual range for day-ahead renewable forecasts.

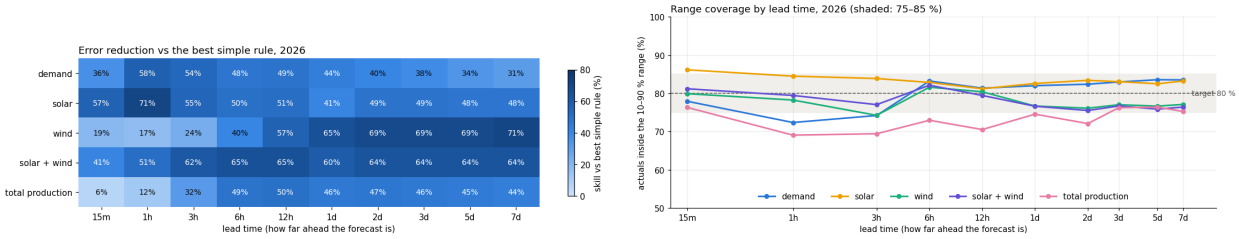


Figure 1: Left: skill against the better simple rule, by series and lead time. Right: coverage of the 10–90 % range, by lead time.

4 Errors and their causes

Total production is the weakest series. Under 1 h it only just beats persistence (6–12 % skill), and its range covers 69–76 % of actual values instead of 80 %. The cause is `other_production`: nuclear, gas, hydro and lignite plants are scheduled by their operators to match the market and exports, and nothing in the features describes those decisions. In some weeks it is off by about 800 MW for days (`reports/week_2026-07-06.png`). Adding the range of total production as if its parts' errors were independent also makes the range too narrow when the errors move together. Adding day-ahead prices as a feature would help most here.

Steady bias in 2026. Beyond one day, demand is over-forecast by about 65 MW and wind is under-forecast by about 60 MW. 2026 had lower demand and more wind than the training years, and the trees learned the earlier level. Training on recent data reduces this, and the final models include 2026.

Short-lead ranges for demand are too narrow. At 1–3 h ahead, demand coverage drops to 72–74 %. The calibration factors come from the last three months of 2025, and 2026 behaves differently, as the bias above shows.

Errors at 3–7 days are probably understated. The weather used in testing comes from archived forecasts made close to each target time. Real 5–7-day weather forecasts are less accurate, so in live use solar and wind errors at those lead times will be noticeably higher. Testing

with Open-Meteo's *Previous Runs* API, which stores forecasts made 1–7 days earlier, would give an honest measurement.

Wind and solar percentages look large. Wind's error is about 27 % of its *mean* output, and solar's about 15 %, because the mean is low (calm periods, nights). Measured against installed capacity, the errors are about 7 % and 3 %.

Accuracy depends on how old the data is. The forecast is issued from the end of the data by default. If `dataset.csv` is not updated, every request is in effect further ahead than it appears, and after 7 days requests are rejected.

National, not site-level. The forecasts are national totals scaled down to one site. They describe the shape of a day well, but not one site's own clouds or load events.